Shwetha

Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal

Turn a rough idea into a precise, buildable project description.

Instructions

Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:

Quiz Application

2. Problem you want to solve (2–3 short sentences):

students often need a simple way to practice questions and test their knowledge. A quiz application helps user answer questions & instantly see their score

3. Target users (who will actually use this?):

Students & learners

4. Inputs (what data / actions will the program receive?):
   - User answer to questions
   - Quiz answer (from file or predefined)
5. Outputs (what will the user see / get?):
   - Score after quiz
   - correct / incorrect answers.
6. Constraints or assumptions (keep it realistic):
   - Limited number of questions
   - Runs on local system.

Part B – Core features vs "nice-to-have"

1. List minimum features needed for a working version (MVP):
   - Display questions
   - Take user answers
   - Calculate score
2. List optional features (only if time allows):
   - Time for quiz
   - Difficulty levels
3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

Taking answer & showing score, because it is the main purpose of a quiz app.

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal
Learn to split a project into logical modules and functions.

Instructions
Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
   - Module 1: data-input
   - Module 2: data-processing
   - Module 3: data-store
   - Module 4: user-interface
2. For each module, write its main responsibility in one sentence:
   - Module 1 does: Take input from user
   - Module 2 does: processes operations
   - Module 3 does: stores data
   - Module 4 does: Display output

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: data-processing.
2. Functions planned:
   - Function 1 name: add-Student()
     - Input(s): name, marks
     - Output: record added
   - Function 2 name: view-students()
     - Input(s): None
     - Output: List of students
   - Function 3 name: delete-student()
     - Input(s): Student ID
     - Output: record removed
3. Draw or briefly describe how data flows between these functions:

user → input → processing → storage → output

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

   Transaction

2. For this entity, list required fields:
   - Field 1: transaction-id (type: int/str/float/...)
   - Field 2: amount (type: float)
   - Field 3: data (type: str)
   - Field 4: type (type: str-credit/debit)

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

```
item = {
  "field1": "transaction_id": 101,
  "field2": "amount": 2500,
  ...
}
```

2. Option B – Class:

python

```
class Item:
  def __init__(self, field1, field2, ...):
    self.field1 = field1
    self.field2 = field2
    ...
```

3. Which option will you use, and why?

I will use Dictionary becau it is simple, easy to implement & suitable for small project

4. Write a short example of creating and using this structure:

python

```
# Example creation and simple usage
transaction["transaction-id"])
print("amount:",
transaction["amount"])
```

```
transaction = {
  "transaction-id": 101,
  "amount": 2500,
  "date": "2026-04-17",
  "type": "credit"
Print ("transaction Id;",
```

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal
Build habit of designing tests before writing code.

Instructions
Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: Calculate_fee
2. In simple words, what should this function do?

This function should calculate the total fee after applying discount based on the original fee amo

3. List at least 5 test cases (inputs and expected outputs):

Test #	Input(s)	Expected output
1	6000	5400
2	5000	5000
3	7000	6300
4	3000	3000
5	5500	4950

4. Now write a small test harness:

python

```
def my_func(...):
  # TODO: implement
  pass

# Tests
print("Test 1:", my_func(...), "expected:", ...)
print("Test 2:", my_func(...), "expected:", ...)
```

5. Reflection:
   - Did writing tests first change how you thought about the function? How?

Yes writing test cases 1st helped identify conditions clearly. & ensured correct outputs before coding.

Activity A5 – Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal

Identify and protect weak points with validation and exception handling.

Instructions

Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Invalid input (user enters text instead of Number)
   - How to detect it? Value error
   - How to handle it politely for the user? Show message: "Invalid input! Please enter a number".
2. Failure Point 2: Empty input
   - Detect: Check if input is empty (if input == "")
   - Handle: Show message: "input can't be empty".
3. Failure Point 3: Negative value (invalid data)
   - Detect: Check if value < 0
   - Handle: Show message: "please enter a positive value."

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")


try:
  with open(file_path, "r") as f:
    data = f.read()
  print("File loaded successfully.")
except FileNotFoundError:
  print("File not found. Please check the path.")
except PermissionError:
  print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

```
# Your robust code:  try:
                        Marks = int(input("enter marks: "))
                        if marks < 0:
                            Print("Please enter a positive value.")
                        Else:
                            print("Marks entered:", marks)
                        except value error:
                            Print("Invalid input! Please enter a
                                number.")
```

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal
Practice cleaning up messy, working code into clear, modular code.

Instructions
Take one working script (from your own work or given by teacher).
It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

Student record Management system- used to store & display student details

2. Identify at least three problems (long function, repetition, confusing names, etc.):
   - Problem 1: Long function (all logic in one place)
   - Problem 2: Repetition of code
   - Problem 3: confusing variable names
3. Plan refactoring steps (high level):
   - Step 1: split code into smaller function
   - Step 2: use meaningful variable and function names
   - Step 3: Remove repeated code
4. After refactoring, write or paste your improved version here:

python

 # Refactored code:

5. Reflection:
   - What became easier to understand after refactoring?

After refactoring the code became easier to understand, more organized, & simpler to modify or debug.

4)
```
def add_student(records, name, marks):
    student={"name": name, "Marks": marks}
    records.append(student)
def display_students(records):
    for student in records:
        print("Name:",
student["name"], "marks:",
student["Marks"])

# Main program
records=[]
add_student(records, "shwetha", 85)
add_student(records, "Anitha", 90)
display_students(record)
```

Activity A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:
   Example:
   - python student_tool.py add --name Alice --score 85
2. What should the tool do in ONE sentence?

The tool allows user to input student name & score & display them through the command line

3. Sketch CLI arguments:

Argument / Option	Meaning
Name	Name of student
Score	Score of the student

4. Implement with argparse (skeleton):

python

```
import argparse

parser = argparse.ArgumentParser(description="Describe your tool here")
parser.add_argument("--name", type=str, help="Student name")
parser.add_argument("--score", type=int, help="Student score")
args = parser.parse_args()

# Use args.name, args.score here
print("Name:", args.name)
print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

quiz_tool.py -- name sahana -- score 85

quiz_tool.py -- name shwetha -- score 90

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal
Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):
   - Milestone 1: Design question on data structure
   - Milestone 2: Load questions from file
   - Milestone 3: Implement quiz flow (ask questions)
   - Milestone 4: calculate & display score
   - Milestone 5: Add basic CLI support
2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date
1	Question format ready	Day 1
2	Question successfully	Day 2
3	Quiz runs with user input	Day 3

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):
   - Risk 1: problem fails to read files
     - Plan B if this fails: use hard coded questions
   - Risk 2: Not enough time to finish features
     - Plan B if this fails: focus only one quiz functionality
   - Risk 3: logical bugs in scoring
     - Plan B if this fails: Add test cases & debug step by step

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

Breaking problem into smaller, using print statement for debugging

2. One thing I now believe about myself as a problem-solver:

I can solve complex problem by approaching them step by step.

3. If I start a new project tomorrow, what will I do differently from day 1?

I plan the structure, write test cases early & divide the project into clear modules before coding.